

智能体应用开发

从 Skills 到 StateGraph：四种智能体技术全解析

2026 金砖赛 · 人工智能应用创新赛项

目录

#	主题	内容
1~3	智能体概述	定义、四种技能全景、架构分层
4~8	OpenClaw Skill	定义、架构、SKILL.md 规范、加载流程、案例
9~11	Python 控制层	三层架构、核心技术、文件处理案例
12~14	Dify 智能体	平台特性、工作流架构、翻译案例
15~17	LangGraph 智能体	概念对比、StateGraph、小乌龟画正方形
18~20	对比与总结	四种技能对比、学习路径、关键回顾

01 · 什么是智能体

智能体 (AI Agent)

“ 能够感知环境、自主决策、执行行动、并从结果中学习的 AI 系统 ”

智能体的四个核心能力

能力	含义	本模块对应技术
感知	理解用户自然语言输入	OpenClaw Skill 意图匹配
决策	根据输入选择执行策略	Dify 条件分支 / LangGraph 路由
行动	调用工具、脚本、API	Python 程序 / Shell 命令
学习	基于反馈优化行为（循环）	LangGraph ReAct 闭环

智能体 vs 传统程序

维度	传统程序	智能体
输入方式	命令行参数 / API	自然语言对话
执行路径	固定流程，if-else 分支	LLM 动态决策，条件路由
工具使用	硬编码函数调用	LLM 自主选择工具并组合
适应性	需修改代码应对新需求	通过 Skill/工作流快速扩展
状态管理	全局变量 / 数据库	TypedDict State（跨节点共享）
循环推理	需手写 while 循环	StateGraph 原生支持

本模块的四种开发方式

OpenClaw Skill (极简) → Python 脚本 (经典) → Dify 工作流 (低代码) → LangGraph 状态图 (高阶架构)

02 · 四种技能全景

从简到繁的智能体技术栈

层级 1: OpenClaw Skill	— 写说明书，让大模型调用命令
层级 2: Python + Shell	— 编写可执行脚本，处理业务逻辑
层级 3: Dify 工作流	— 可视化拖拽编排 LLM + 工具链
层级 4: LangGraph 状态图	— 代码级图引擎，构建循环推理智能体

核心关系

“ Skill 告诉大模型「什么时候调什么命令」 → Shell 启动环境 → Python 执行业务逻辑 → Dify/LangGraph 提供高阶编排能力 ”

02 · 四种技能全景

选型速查

需求	推荐技术	上手时间
让智能体执行简单命令	OpenClaw Skill	0.5 小时
数据处理、文件操作、API 调用	Python 脚本	1 小时
快速搭建对话机器人	Dify 工作流	1 小时
多步骤推理、循环决策	LangGraph	2 小时
ROS2 机器人控制、传感器闭环	LangGraph + ROS2	3 小时

03 · 智能体架构分层

三层架构

理解层

OpenClaw Skill / Dify LLM / LangGraph Node

自然语言 → 意图识别 → 命令/工具选择

封装层

Shell 脚本 / Dify 变量传递 / StateGraph 边

环境初始化 → 参数组装 → 数据传递

执行层

Python 程序 / Dify 代码节点 / 工具函数

业务逻辑 → API 调用 → 结果输出

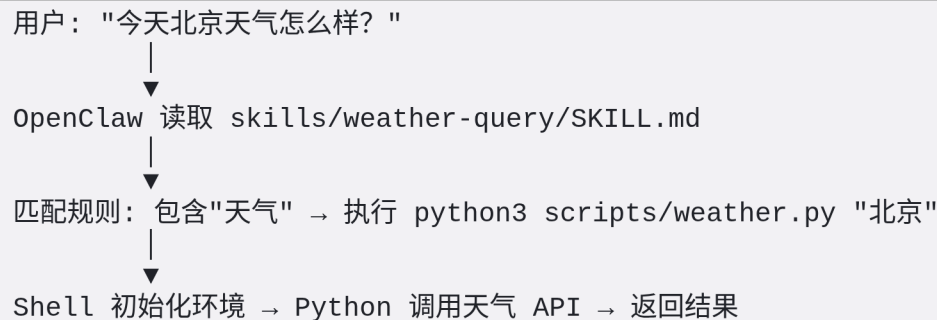
“ 关键原则：各层职责分离——理解层不关心怎么执行，执行层不关心怎么选择 ”

04 · OpenClaw Skill 定义

OpenClaw Skill — 智能体的"技能说明书"

“ 一个 SKILL.md 文件 = YAML 元数据 + Markdown 指令映射表 ”

核心机制

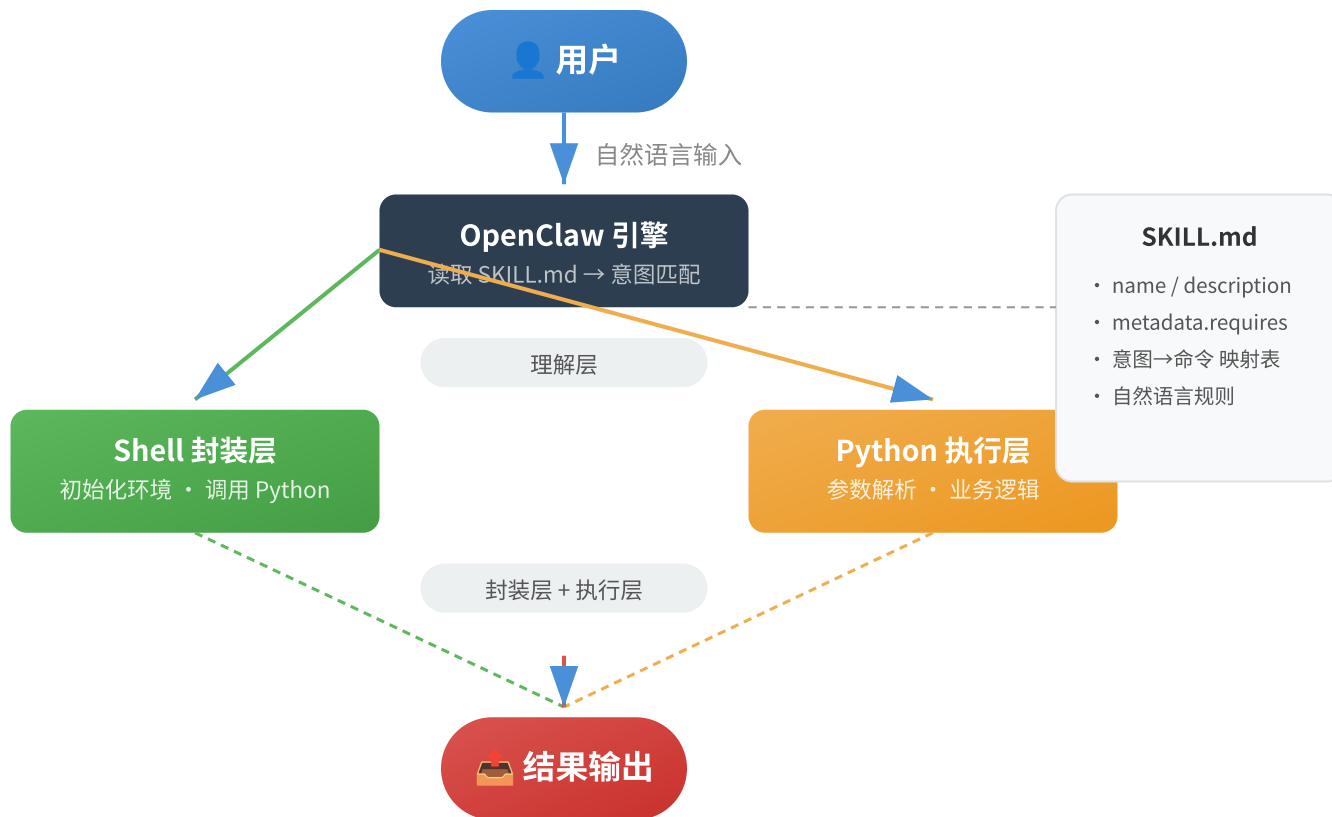


Skill 的三个要素

要素	格式	作用
元数据	YAML Frontmatter	name、description、requires
指令映射表	Markdown 表格	自然语言 → Shell 命令
匹配规则	自然语言描述	告诉大模型何时选择哪条命令

05 · OpenClaw 三层架构

理解 → 封装 → 执行



各层职责

06 · SKILL.md 规范说明

YAML Frontmatter 核心字段

字段	类型	必填	说明
name	string	✓	技能名称，如 <code>weather-query</code>
description	string	✓	技能简述，UI 搜索摘要
version	string		语义化版本号
metadata.openclaw.requires.env	string[]		必要的环境变量
metadata.openclaw.requires.bins	string[]		必要的 CLI 工具
metadata.openclaw.primaryEnv	string		主凭证变量名
metadata.openclaw.always	boolean		设为 true 始终激活
metadata.openclaw.emoji	string		技能图标
metadata.openclaw.os	string[]		操作系统限制
metadata.openclaw.install	array		依赖安装规格

文件结构

```
skills/<skill-name>/
├── SKILL.md
├── .clawhubignore
└── references/
```

- ← 必需
- ← 可选：发布忽略
- ← 可选：参考文档

07 · SKILL.md 完整模板

```
---
name: <skill-name>
description: <一句话描述>
version: 1.0.0
metadata:
  openclaw:
    requires:
      env: [<REQUIRED_ENV>]
      bins: [<required-binary>]
    primaryEnv: <MAIN_VAR>
    emoji: "🦾"
---

# <技能标题>

## 前置条件
- 已配置 <ENV_VAR> 环境变量
- 系统已安装 <required-binary>

## 使用方法
`bash`
<command-example>
`

## 命令映射表
| 用户意图 | 执行命令 | 效果 |
|-----|-----|-----|
| <意图A> | `<命令A>` | <效果> |

## 自然语言 → 命令规则
- **包含"<关键词>"** → 执行 `<command>`
- **默认行为** → <默认处理>

## 注意事项
- <错误处理提示>
```

08 · Skill 加载流程

多来源 → 门控 → 注入

OpenClaw Skill 加载流程

(优先级从高到低)



关键机制

同名覆盖

高优先级同名 Skill
覆盖低优先级

目录不决定名称

Skill 名称来自
frontmatter 的 name 字段

Token 开销

每个 Skill ≈ 24 tokens
基本开销 ≈ 195 chars

会话快照复用

会话启动时快照
同一会话复用
文件变更时刷新

⚠ 第三方 Skill 视为不受信任代码

① 门控过滤 (Gating)

requires.bins / env / config / platforms 筛选

② Allowlist 可见性控制

按智能体限制 → 生成有效 Skills 快照

③ 运行时环境注入

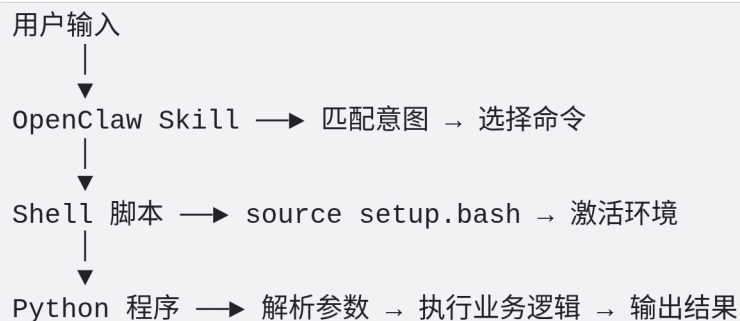
env → API Key → 系统提示词(XML)

④ 智能体执行任务

LLM 读取 Skill 指令 → 匹配命令 → 执行

09 · Python + OpenClaw 控制

三层架构中的执行层



核心技术点

概念	说明	Python 实现
命令行参数解析	接收用户输入	<code>sys.argv</code> / <code>argparse</code>
API 调用	调用外部 HTTP 服务	<code>requests</code> 库
文件 I/O	读写本地文件	<code>Path</code> / <code>open()</code>
进程调用	执行外部命令	<code>subprocess</code>

10 · 文件批量处理案例

file_tool.py — 双命令工具

```
# 子命令: rename — 批量重命名
python3 file_tool.py rename --dir ./data --type txt --prefix done

# 子命令: stats — 统计文件分布
python3 file_tool.py stats --dir ./data
```

函数拆解

函数	功能	核心技术
rename_files(dir, type, prefix)	给指定类型文件加前缀	Path.glob() → Path.rename()
count_files(dir)	按后缀统计文件数	Path.iterdir() → 字典累加
main()	argparser 子命令分发	add_subparsers()

对应 Skill 映射

用户意图	执行命令
批量重命名 / 加前缀	python3 file_tool.py rename --dir ./data --type txt --prefix done
统计文件分布 / 统计	

11 · Python 脚本技术要点

编写规范

要点	说明	示例
参数解析	简单用 <code>sys.argv</code> ，复杂用 <code>argparse</code>	<code>argparse.ArgumentParser()</code>
错误处理	<code>try/except</code> 捕获异常，友好提示	<code>sys.exit(1)</code> 表示失败
输出格式	终端用 <code>print</code> ，结构化用 JSON	<code>json.dumps(result)</code>
环境变量	敏感信息不硬编码	<code>os.environ.get("API_KEY")</code>
Shebang	首行声明解释器	<code>#!/usr/bin/env python3</code>
Docstring	文件头写用法说明	<code>"""file_tool.py - ..."""</code>

关键设计模式

```
argparse 子命令模式：
parser.add_subparsers()
├─ rename_parser → 处理 rename 子命令
└─ stats_parser  → 处理 stats 子命令

错误即退出：
if not path.exists():
    print("[ERROR] ..."); sys.exit(1)
```

12 · Dify 智能体

什么是 Dify

低代码 LLMOps 平台 — 可视化编排 AI 应用

特性	说明
可视化编排	拖拽节点搭建工作流，无需写前端
LLM 集成	支持 OpenAI、通义千问、DeepSeek 等
工具/插件	自定义 API 工具、代码执行节点
知识库	上传文档作为 RAG 知识源
一键发布	内置 Web 应用界面，直接分享

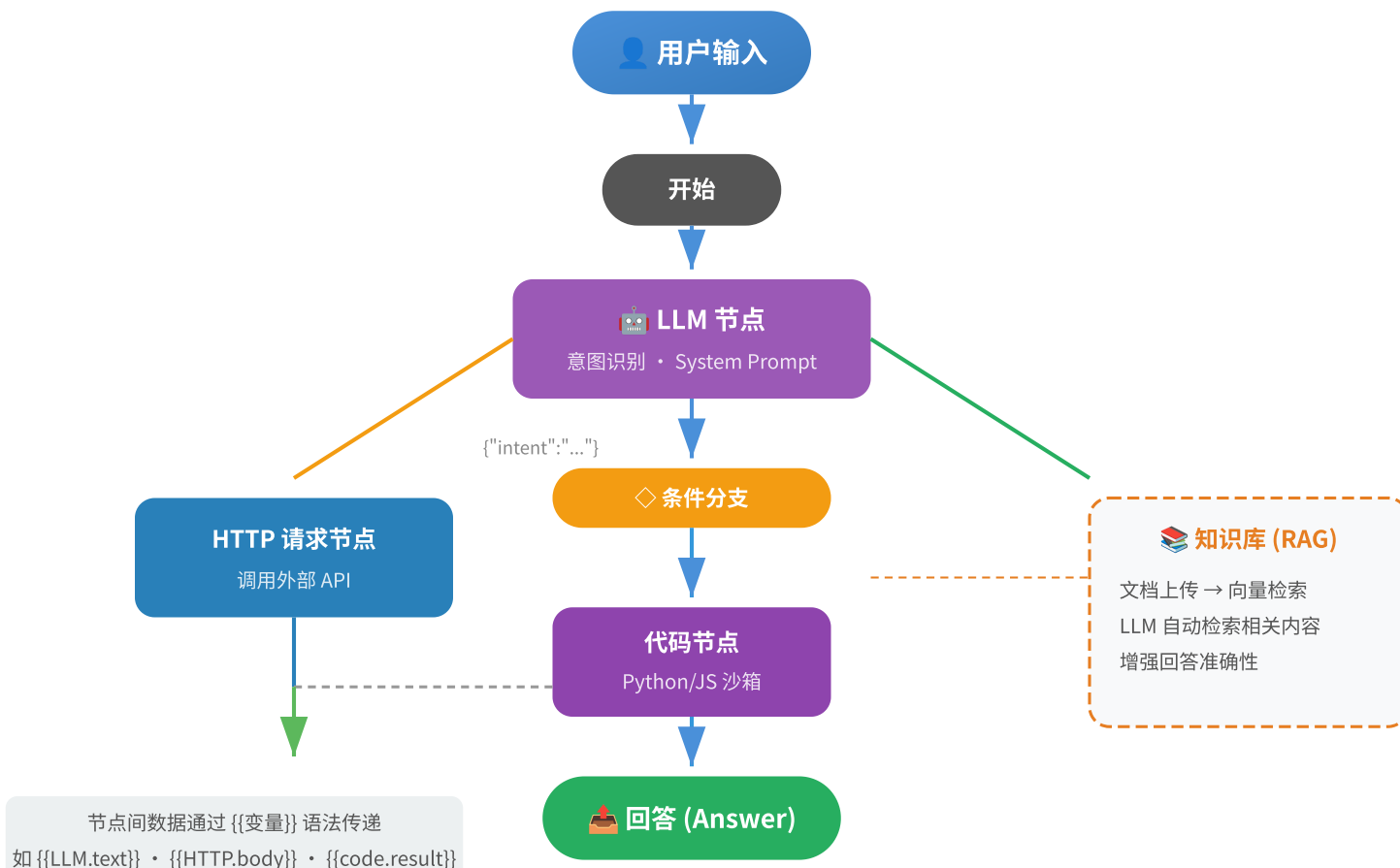
Dify 在智能体中的地位

OpenClaw Skill (文件级说明书) 简单命令映射	vs	Dify 工作流 (可视化工作流) 复杂多步推理
--------------------------------------	----	--------------------------------

13 · Dify workflows 架构

Chatflow 核心节点

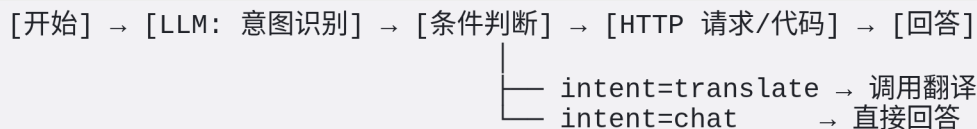
Dify Chatflow 工作流架构



节点类型

14 · 智能翻译 workflow 案例

场景：多语言翻译助手



LLM 节点 System Prompt

你是一个多功能助手。根据用户输入，输出严格的 JSON：

1. 翻译：{"intent": "translate", "text": "你好", "target_lang": "英文"}
2. 聊天：{"intent": "chat", "reply": "你的回复"}

变量传递语法

语法	含义
<code>{{LLM.text}}</code>	引用 LLM 节点的 text 输出字段
<code>{{LLM.target_lang}}</code>	引用目标语言字段
<code>{{HTTP.body.translated}}</code>	引用 HTTP 请求的返回结果

15 · LangChain vs LangGraph

为什么要用 LangGraph

特性	LangChain	LangGraph
定位	LLM 应用开发框架	有状态图计算引擎
核心抽象	Chain ($A \rightarrow B \rightarrow C$ 链式)	StateGraph (可循环、可分支)
控制流	线性固定顺序	循环 + 条件动态路由
记忆	会话记忆 Buffer	全局 State (TypedDict 共享)
适合场景	简单问答、RAG、工具调用	多步骤推理、闭环控制

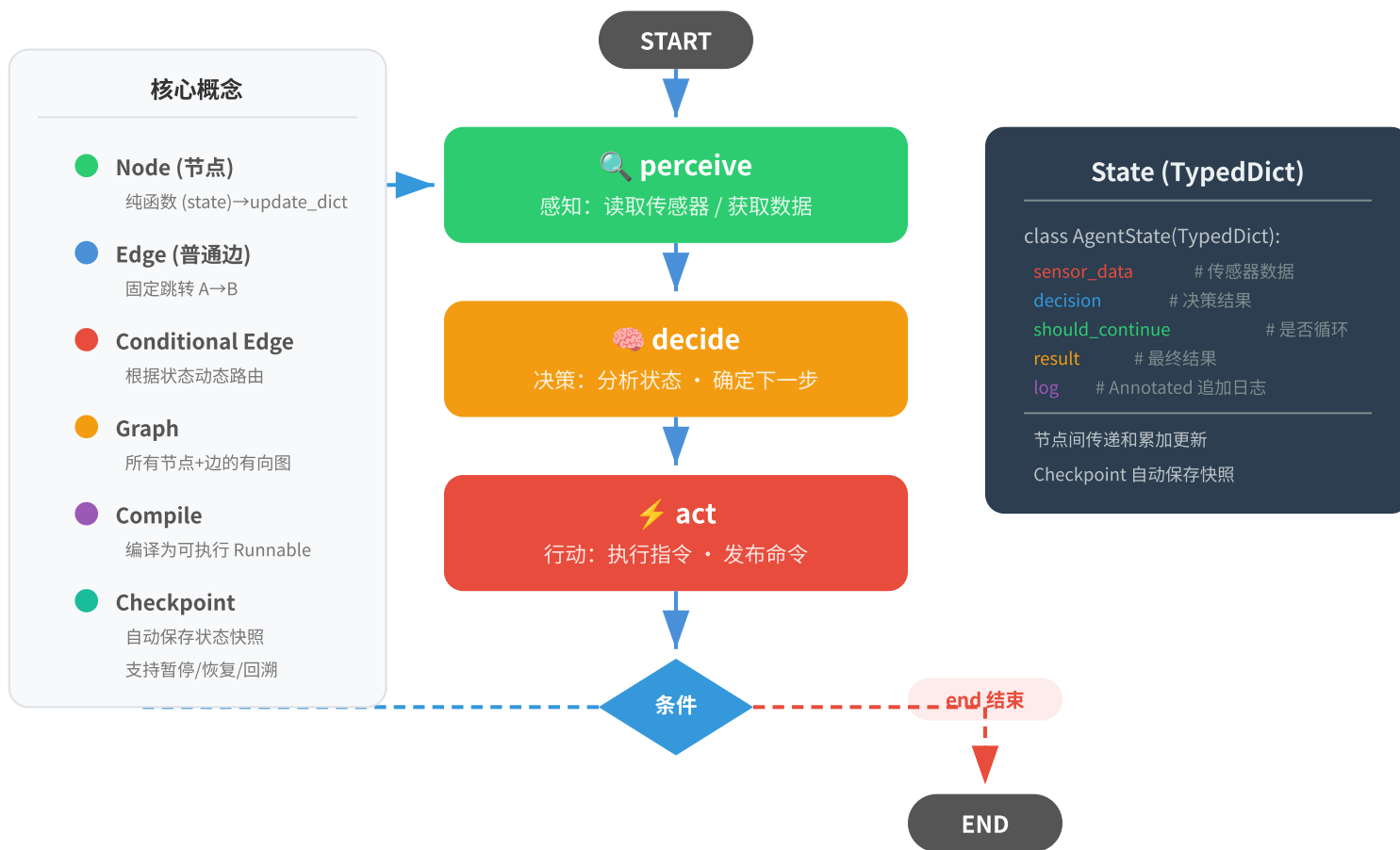
为什么智能体需要 Loop

“ 智能体的本质是 **思考** → **行动** → **观察** → **思考** 的闭环。普通 Chain 只能 $A \rightarrow B \rightarrow C$ 一次性跑完，无法「做了再想、想了再做」。
StateGraph 天生支持循环，是构建真正智能体的基础。 ”

16 · LangGraph 核心概念

StateGraph 五要素

LangGraph StateGraph 架构



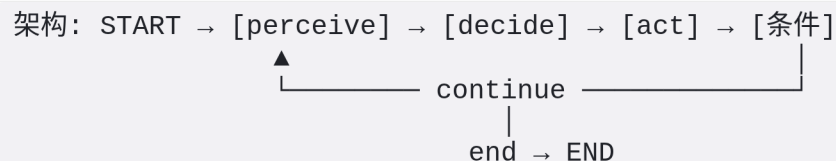
17 · 小乌龟画正方形案例

LangGraph + ROS2 入门

“

前置条件: `ros2 run turtlesim turtlesim_node` 启动仿真

”



State 定义

```
class TurtleState(TypedDict):
    pose_x: float          # 当前位置
    pose_y: float          # 当前位置
    pose_theta: float      # 当前朝向
    side_count: int        # 已完成边数
    step: str              # "forward" | "turn" | "done"
    linear_x: float        # 当前线速度
    angular_z: float       # 当前角速度
    log: Annotated[list[str], operator.add]
```

17b · 小乌龟画正方形（续）

三节点设计

节点	功能	输出
perceive	读取 turtlesim 位姿话题	pose_x, pose_y, pose_theta
decide	判断该直走还是转弯	step: "forward" / "turn" / "done"
act	发布 /turtle1/cmd_vel 速度指令	linear_x, angular_z

条件路由 route_after_act

```
def route_after_act(state) -> str:
    if state["step"] == "done":
        return "end"          # → END, 停止
    return "continue"         # → perceive, 继续循环
```

关键配置

```
CFG = {
    "forward_speed": 2.0,      # 直走 2.0 m/s
    "forward_time": 1.8,       # 持续 1.8 秒 ≈ 边长
    "turn_speed": 1.57,        # 转弯 ≈  $\pi/2$  rad/s
    "turn_time": 1.0,          # 转 1.0 秒 ≈ 90°
    "total_sides": 4,          # 正方形 = 4 条边
}
```

18 · 四种技能对比

维度	OpenClaw Skill	Python 控制	Dify 智能体	LangGraph
开发难度	★ 极低	★★ 中等	★ 低	★★★ 较高
代码量	~50 行 MD	~80 行 Python	~30 行 + GUI	~150 行 Python
自然语言交互	✓ 原生	✗ 需自己解析	✓ 可视化编排	✓ 集成 LLM
循环控制	间接（重调脚本）	✓ 直接	间接（条件分支）	✓ 节点内实现
可视化调试	✗	✗	✓ 工作流视图	✓ 图可视化
适合场景	简单命令映射	数据处理	快速原型	复杂多步推理
外部 API	✗ 通过 Shell	✓	✓	✓
学习价值	理解智能体交互	理解程序控制	理解 LLM Ops	理解 Agent 架构

结论

“ Skill 入门最快，Python 最实用，Dify 最快出原型，LangGraph 天花板最高——四种技术互补而非替代 ”

19 · 学习路径

入门阶段 · 建立直觉

- 1 OpenClaw Skill → 理解「用户 → 智能体 → 命令」链路
- 2 Python 脚本 → 理解参数解析、文件 I/O、API 调用

进阶阶段 · 低代码编排

- 3 Dify 工作流 → 理解可视化编排、LLM 意图识别

高阶阶段 · 架构思维

- 4 LangGraph 智能体 → 理解 StateGraph + 循环推理
 - 5 ROS2 集成 → 控制真实/仿真机器人

建议用时

阶段	技能	核心收获	建议用时
入门	OpenClaw Skill	掌握 SKILL.md 编写	30 分钟
入门	Python 脚本	命令行参数、文件操作、API	1 小时
进阶	Dify 工作流	工作流编排、意图识别、工具注册	1 小时
高阶	LangGraph	StateGraph、条件路由、循环推理	2 小时

课程总结

你已掌握的技能

环节	技能点
 Skill 编写	SKILL.md 规范、YAML Frontmatter、命令映射表
 Python 控制	argparse、文件 I/O、API 调用、subprocess
 Dify 工作流	可视化编排、LLM 意图识别、条件分支、变量传递
 LangGraph	StateGraph、节点函数、条件路由、ReAct 循环
 ROS2 集成	turtlesim 控制、传感器闭环、StateGraph 机器人

关键技术要点回顾

四个核心理解

1. **Skill 是说明书，不是代码** — 它告诉智能体「选哪条命令」，而非执行逻辑
2. **三层分离** — 理解层、封装层、执行层各司其职，绝不耦合
3. **StateGraph > Chain** — 循环推理是智能体的本质需求，StateGraph 提供了正确的抽象
4. **从简到繁** — Skill → Python → Dify → LangGraph，每种适合不同复杂度

核心技术路线

```
OpenClaw SKILL.md 说明
  → Shell 环境封装
    → Python 业务逻辑
      → Dify 低代码编排
        → LangGraph 状态图循环推理
          → ROS2 机器人闭环控制
```

参考资料

- **OpenClaw** — <https://docs.openclaw.ai/>
- **AgentSkills 规范** — <https://agentskills.io>
- **Dify** — <https://dify.ai/>
- **LangChain** — <https://www.langchain.com/>
- **LangGraph** — <https://langchain-ai.github.io/langgraph/>
- **ROS2** — <https://docs.ros.org/en/humble/>
- **turtlesim** — ROS2 内置教学仿真包

Thank You

准备好迎接智能体开发的全新范式了吗？

从零开始只需三步：

1. 编写第一个 SKILL.md
2. 创建对应的 Python 脚本
3. 在 OpenClaw 中说一句话，看它自动执行！

“

 配套文档：

- 模块4-智能体应用开发-基础知识
- 模块4-智能体应用开发-实操手册
- 智能体应用开发技术架构说明

”